

CMPUT274—Assignment 2 (Fall 2026)

R. Hackman

Due Date: Friday October 9th, 8:00PM

Per course policy you are allowed to engage in *reasonable* collaboration with your classmates. You must include in the comments of your assignment solutions a list of any students you collaborated with on that particular question.

For each assignment you may not use any Python features or functions not discussed in class! This means, for example, you may not use loops, casting, any built-in functions or methods we have not discussed. If you are in doubt, you may ask on the discussion forum — however remember if you are including your code, or even your plan to solve a question, you must make a private post. Use of disallowed features will result in a grade of zero on the question(s) in which they were used.

All functions you write must have a complete function specification as presented in the course notes. Failure to provide function specifications for functions will lead to the loss of marks.

Unless explicitly forbidden by a question, you are always allowed to write additional helper functions that may help you solve a problem. In fact in some cases it will be *necessary* to write helper functions!

Allowed Keywords and Functions

· True · False · def · and · assert · if · elif · else · import · from · not ·
or · return · testExact · testWithin · Arithmetic operators
· String subscription, slicing, and concatenation · foldr · foldl · map ·
filter · lambda · LL

1. **LList Cleaning** in this question you will write the Python function `removeOccurrences` which has two parameters. The first parameter, `l`, is an **LList** of any values. The second parameter, `toRemove`, is any value. Your function must return a **LList** which is the result of removing all occurrences of `toRemove` from `l`. For example:

```
removeOccurrences(LL(1, 5, 10, 22, 10, 33), 10) -> (1, 5, 22, 33)
```

You are again provided a skeleton file `q1.py` to fill in. This provides some basic test cases already written for you, as well as empty function definition. While there are two basic test cases provided, they are insufficient to effectively test your function. In addition to filling in the function definition you should write your own test cases to be confident your solution works.

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. To make a new directory and copy the file for this question into that new directory you can follow the commands below:

```
cd
mkdir myA2
cd myA2
cp ~/f26-cmput274/a2/provided/q1.py ./
```

Deliverables: For this question include in your final submission zip the python file `q1.py`

2. **Splitting Strings** In this question you will be writing a function named `strSplit` which takes two parameters, both of which are strings, and returns a `LList` of strings which is the result of “splitting” the string on all occurrences of the given separator. For this question the function specification is provided for you below.

```
def strSplit(s, sep):
    '''
    strSplit splits the string s on all occurrences of sep
    and returns a LList of the substrings of s
    that are separated by sep

    s          - a string
    sep         - a string
    returns    - LList of strings

    Examples:
    strSplit("123,abcdf,hello", ",") -> ("123", "abcdf", "hello")
    strSplit("no split found", ",") -> ("no split found")
    strSplit("no split found", " ") -> ('no', 'split', 'found')
    strSplit("HelloxyFriendxHowxyAreYou?", "xy") -> ('Hello', 'FriendxHow', 'AreYou?')
    '''
    ...
```

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. If you have followed all instructions in the previous questions so far, then you can make a copy of this file by running the commands below:

```
cd ~/myA2
cp ~/f26-cmput274/a2/provided/q2.py ./
```

Deliverables: For this question include in your final submission zip the python file `q2.py`

3. **Integer detection** In this question you will write the function `isIntegerStr` which takes one string parameter and returns `True` if that string represents an integer and `False` otherwise. A string represents an integer if:

- All the characters in the string are digit characters
- Except, optionally, the first character which may be the `-` character instead of a digit.

For example:

```
isIntegerStr("456") -> True
isIntegerStr("954x6") -> False
```

Hint: How can the `ord` function help you detect if a character is a digit or not?

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. If you have followed all instructions in the previous questions so far, then you can make a copy of this file by running the commands below:

```
cd ~/myA2
cp ~/f26-cmput274/a2/provided/q3.py ./
```

Deliverables: For this question include in your final submission zip the python file `q3.py`

4. **String to Integer** In this question you will write the function `strToInt` which takes one string parameter that is guaranteed to be a string that represents an integer. Your function must return the corresponding `int` that the string represents. For example:

```
strToInt("-432") -> -432
strToInt("1234") -> 1234
```

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. If you have followed all instructions in the previous questions so far, then you can make a copy of this file by running the commands below:

```
cd ~/myA2
cp ~/f26-cmput274/a2/provided/q4.py ./
```

Deliverables: For this question include in your final submission zip the python file `q4.py`

5. **Hamming Distance** The *Hamming distance* between two strings is the number of positions in which their characters differ. Typically, Hamming distance is defined between two strings of the same length, however we will define it for two arbitrary strings. In the case where one string is larger than the other, then each character the longer string is longer by is also considered a character in the distance, as it is a position in which the strings differ (one string has a character in that position, while the other does not).

For example

```
hammingDistance("xyz", "axzz") -> 1
hammingDistance("abc----", "abc") -> 4
hammingDistance("grey", "hazy") -> 3
```

You must write the function `hammingDistance` such that takes two strings and produces the natural number that is the Hamming distance between those two strings.

Restriction: You may not write *any* functions that are explicitly recursive. Instead of using any recursion you must use higher order functions to complete this problem. Solving the problem using recursion would result in no marks awarded.

You are again provided a skeleton file `q1.py` to fill in. This provides some basic test cases already written for you. While there are three basic test cases provided, they are insufficient to effectively test your function. In addition to filling in the function definition you should write your own test cases to be confident your solution works.

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. To make a new directory and copy the file for this question into that new directory you can follow the commands below:

```
cd
mkdir myA3
cd myA3
cp ~/f26-cmput274/a2/provided/q5.py ./
```

Deliverables: For this question include in your final submission zip the python file `q5.py`

6. **Deduplication** The act of *deduplicating* a sequence is the act of creating a new sequence where only the *first* occurrence of each value appears in the resultant sequence.

Write the function `dedup` that takes a single `LList` parameter and returns the `LList` that is the result of deduplicating the given input `LList`. For example:

```
dedup(LL(1, 1, 1, 2, 1, 8, 2, 7, 8)) -> (1, 2, 8, 7)
dedup(LL(2, 3, 5, 7, 9, 9, 7, 9, 5, 2, 3, 2, 7)) -> (2, 3, 5, 7, 9)
```

Restriction: You may not write *any* functions that are explicitly recursive. Instead of using any recursion you must use higher order functions to complete this problem. Solving the problem using recursion would result in no marks awarded.

Once again, you should not edit any files in the repository itself, but instead make your own copy of it to edit. If you have followed all instructions in the previous questions so far, then you can make a copy of the provided starter file by running the commands below:

```
cd ~/myA2
cp ~/f26-cmput274/a2/provided/q6.py ./
```

Deliverables: For this question include in your final submission zip the python file `q6.py`